nabia Python: Package, Examples and Reference

Research on the nabia C++ library from Python

nabia documentation

October 2026

Abstract

This document is the reference for the Python project of nabia (`python/`): the `nabia` package and the examples, built on the `nabia_python_bridge` wheel of the C++ project. It covers:

- installing the wheel and using the whole C++ library from Python;
- portfolio backtests on pandas data (`nabia.backtest`, `nabia.latest_weights`);
- the analytics computed on price frames (`nabia.analytics`): correlations, volatilities, trend signals, drawdowns and the market regime;
- the market data pipeline (`nabia.marketdata`);
- six worked examples on market data, which together use every function of the bridge.

The methods themselves are derived in the C++ reference (`cpp/doc`); the agents and the web application that use this package are in the web UI reference (`webui/doc`). Excerpts and code locations are extracted from the source when the document is built.

Contents

I Overview **3**

1 Purpose and layout **3**
1.1 Three projects . . . 3
1.2 Directory map . . . 4
1.3 Environment . . . 4

II The C++ library in Python **4**

2 `nabia_python_bridge` **4**
2.1 Modules . . . 4

3 Portfolio backtests: `nabia.backtest` **5**

III Data and analytics **6**

4 Market data **6**
4.1 Universe and sources . . . 6
4.2 Storage and incremental updates . . . 7

5 Analytics on price frames **8**

6 Worked examples on market data **10**

IV Verification and limitations **11**

7 Verification **12**

8 Limitations **12**

Part I
Overview

1 Purpose and layout

The Python project is where nabia is used for research: it loads market data, runs the C++ engines on pandas objects and plots the results. It contains no C++: the library comes from the `nabia_python_bridge` wheel, built by the C++ project into `dist/`.

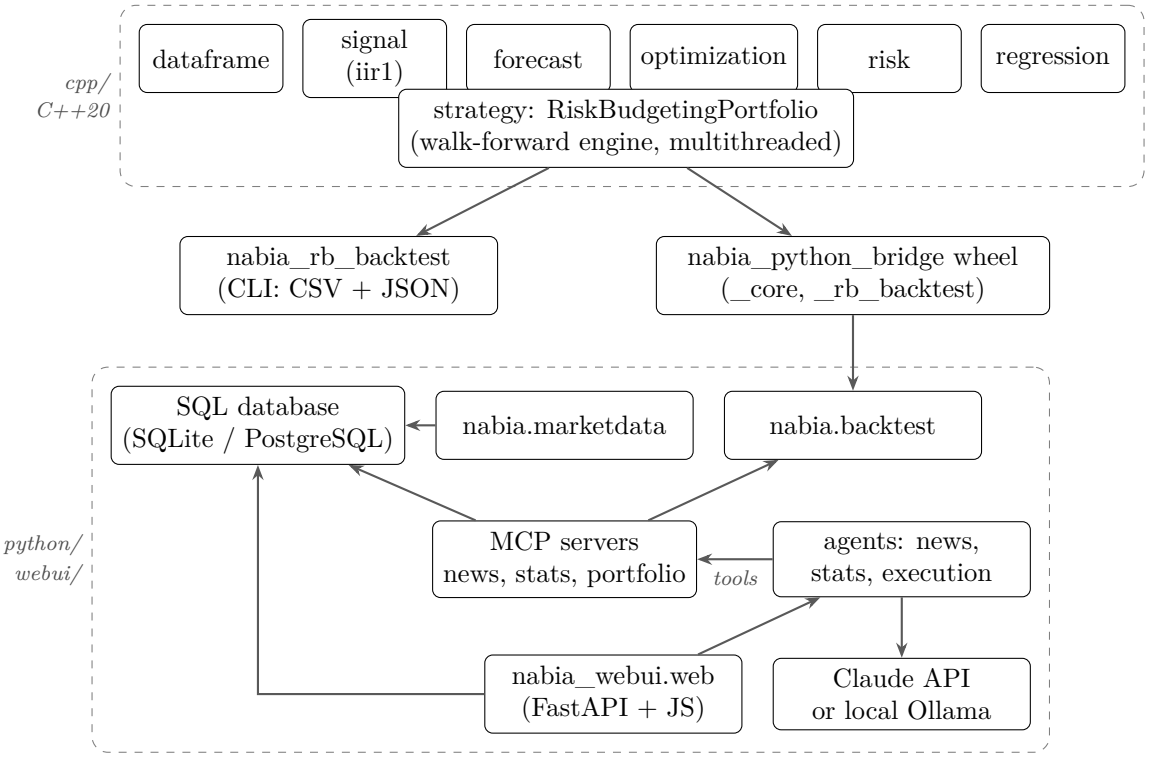


Figure 1: The whole system and its three projects. Arrows point from a component to what it uses.

1.1 Three projects

The repository holds three independent projects. Each opens on its own in an IDE, and each has its own documentation in a `doc` folder:

Folder	IDE	Contents
`cpp/`	CLion (CMake)	the C++ library (`include`, `src`), its tests, the command-line application, and the pybind11 bindings that build the `nabia_python_bridge` wheel into `dist/`
`python/`	PyCharm	the `nabia` package (backtests, analytics, market data) on top of the wheel, the examples, and the Python tests including the bridge tests
`webui/`	PyCharm	`nabia_webui`: the agents, their MCP servers and the web app, on top of the `nabia` package

The projects depend on each other only through installed packages: `cpp` builds the wheel, `python` installs it (`python/scripts/setup_env.sh`), and `webui` installs both the wheel and `python` in editable mode (`webui/scripts/setup_env.sh`). Shared data, the market database `data/market.db`, is at the repository root.

1.2 Directory map

Path (in `python/`)	Contents
`nabia/portfolio.py`	`backtest`, `latest_weights` on pandas prices (§3)
`nabia/analytics.py`	statistics on price frames (§5)
`nabia/marketdata`	universe, sources, SQL store, incremental download (§4)
`examples`	the worked examples and their data and plotting helpers (§6)
`tests`	pytest suites; `tests/bridge` tests the wheel itself
`scripts/setup_env.sh`	creates `.venv` with the wheel and this package
`doc`	this reference (LaTeX source, PDF, `build.sh`)

1.3 Environment

```
cpp/scripts/build_wheels.sh 3.14          # in the C++ project: the wheel, into dist/
python/scripts/setup_env.sh               # python/.venv: the wheel + nabia (editable) + extras
cd python && .venv/bin/python -m pytest   # 49 tests, 2 opt-in network tests skipped
```

In PyCharm, open `python/` and select `python/.venv` as the interpreter. After the wheel is rebuilt, run `setup_env.sh` again: it reinstalls the wheel every time. To debug the C++ code under a Python script, build the wheel with `NABIA_WHEEL_BUILD_TYPE=RelWithDebInfo` and attach CLion's debugger to the Python process.

Part II
The C++ library in Python

2 `nabia_python_bridge`

The wheel exposes every public class and function of the C++ library. How it is built, and how the bindings handle parameters, threads and errors, is in the C++ reference (*The full library in Python* (C++ reference)).

2.1 Modules

Module	Contents
`stats`	`mean`, `variance`, `stddev`, `autocorrelation`; `normal_pdf/cdf/quantile` (vectorized)
`signal`	`EWMA`, `ButterworthBandPass` (streaming `process`, batch `apply`), `moving_average`, `exponential_moving_average`, `diff`
`forecast`	`KalmanFilter` (local level, local linear trend, custom), `GaussianProcess` with squared exponential, Matérn 5/2, periodic, sum and product kernels (`k1 + k2`, `k1 * k2`), `SimpleExponentialSmoothing`, `Forecast`
`optimization`	`LBFGS`, `AugmentedLagrangian`, `SQP`, `NelderMead`, `DifferentialEvolution` with Python objectives; `BoxConstraints`, `LinearConstraint`
`risk`	`Volatility`, `ValueAtRisk`, `ExpectedShortfall`, `RiskBudgeting`
`strategy`	`RiskBudgetingPortfolio`, `compute_performance`, `equity_curve`, `drawdown_series`
`regression`	`ElasticNet`, `elastic_net_path`, `lambda_max`, `lambda_grid`, `RegressionEngine` (*Elastic net factor regression* (C++ reference))
`frames`	pandas interface: `backtest`, `regress`, `performance`, `metrics_dict`

The `TimeSeries` and `DataFrame` containers are not bound, because pandas does their job in Python. Where an engine takes indexed data, `frames` maps the pandas index to row positions and back.

Conventions.

- Arrays are numpy `float64`: vectors are 1-d, and matrices have one row per observation.
- Parameter structs take their fields as keyword arguments, and unknown names raise `AttributeError`: `signal.EWMAParams(alpha=0.2)`. Models also accept the fields directly: `signal.EWMA(alpha=0.2)`. `lambda` is spelled `lambda_`.
- Enums are Python enums (`regression.SubsetSearch.Forward`); in `frames`, a member name also works (`risk_measure="ValueAtRisk"`).
- `std::invalid_argument` becomes `ValueError`, "not fitted" becomes `RuntimeError`, and an exception raised in a Python objective propagates out of `optimize()`.
- Fits, backtests, regressions and optimizations release the GIL; Python objectives take it back for each call, so `DifferentialEvolution(num_threads > 1)` is safe.

```
import nabia_python_bridge as nb
from nabia_python_bridge import frames

rb = nb.risk.RiskBudgeting(nb.risk.RiskBudgetingParams(budgets=[0.5, 0.3, 0.2]), nb.risk.Volatility(cov
    ↪ ))
weights = rb.optimize()
reg = frames.regress(fund_returns, factor_returns, min_factors=1, max_factors=3, train_fraction=0.7)
```

3 Portfolio backtests: `nabia.backtest`

`nabia.portfolio` runs the C++ portfolio engine through `_rb_backtest`, a module of the wheel's package `nabia_python_bridge`. The module reuses the command-line application's JSON parser: Python and the command line accept the same parameters with the same error messages (*Backtest bindings for the Python package* (C++ reference)). Data crosses the boundary in simple forms: dates as `int64` days since 1970-01-01, prices as a `float64` matrix, and parameters as JSON text.

Two functions are exposed:

- `nabia.backtest(prices, params, budget_schedule=None)` returns a result with pandas frames and metrics;
- `nabia.latest_weights(prices, params)` solves only the last rebalance. The agents of the web UI use it to turn budgets into target weights.

Code 1: pandas to C++
`nabia/portfolio.py:103-109`

```
def _inputs(prices: pd.DataFrame, params: Mapping[str, Any] | None) -> tuple[np.ndarray, list[str], np.
    ↪ ndarray, str]:
    if not isinstance(prices, pd.DataFrame):
        raise TypeError("prices must be a pandas DataFrame (dates x assets)")
    if prices.shape[1] == 0:
        raise ValueError("prices has no columns")
    values = np.ascontiguousarray(prices.to_numpy(dtype=np.float64, na_value=np.nan))
    return _to_days(prices.index), [str(c) for c in prices.columns], values, _config_json(params)
```

Code

Python `backtest` — `nabia/portfolio.py:112-182`
Python `latest_weights` — `nabia/portfolio.py:185-202`
dates to day numbers — `nabia/portfolio.py:72-80`

Part III
Data and analytics

4 Market data

4.1 Universe and sources

The default download universe has four asset classes:

- equity indices: S&P 500, Nasdaq 100, Russell 2000, Euro Stoxx 50, DAX, FTSE 100, Nikkei 225, Hang Seng;
- US Treasury futures: 2Y, 5Y, 10Y, Bond, Ultra Bond;
- commodity futures: WTI, Brent, natural gas, gold, silver, copper, corn, wheat, soybeans;
- crypto against USD: BTC, ETH, SOL, XRP, ADA, LTC.

The AI portfolio uses the nine assets of Table 1.

Table 1: Default AI portfolio universe.

Symbol	Asset	Class
`^GSPC`	S&P 500	equity index
`^STOXX50E`	Euro Stoxx 50	equity index
`^N225`	Nikkei 225	equity index
`ZN=F`	US 10-Year T-Note future	bond future
`ZB=F`	US Treasury Bond future	bond future
`BZ=F`	Brent crude oil future	commodity
`GC=F`	Gold future	commodity
`HG=F`	Copper future	commodity
`BTC/USD`	Bitcoin	crypto

Sources.

- **Yahoo Finance** (`yfinance`) provides indices and futures, in batches. Futures are front-month continuous series, which jump at contract rolls. WTI settled negative on 2020-04-20, where simple returns are undefined, so the portfolio uses Brent.
- **Crypto exchanges** (`ccxt`, Coinbase by default) provide daily candles from public data, with no key. History is paged in windows of 300 candles. Windows before a coin's listing date are empty and are stepped over. Paging stops if the exchange ignores the start date.

Code 2: Crypto daily candles with pagination
`nabia/marketdata/sources.py:147-188`

```
def fetch(self, symbols: Sequence[str], start: dt.date, end: dt.date | None = None) -> pd.DataFrame:
    client = self._exchange()
    start_ms = int(dt.datetime.combine(start, dt.time(), dt.UTC).timestamp() * 1000)
    last_day = end or dt.datetime.now(dt.UTC).date()
    stop_ms = int(dt.datetime.combine(last_day + dt.timedelta(days=1), dt.time(), dt.UTC).timestamp() *
    ↪  1000)
    frames = []
    for symbol in symbols:
        if symbol not in client.markets:
            log.warning("%s is not listed on %s", symbol, getattr(client, "id", "the exchange"))
            continue
```

```
        candles: dict[int, list[float]] = {}
        since = start_ms
        while since < stop_ms:
            page = self._page(client, symbol, since)
            if not page:
                # Nothing in this window: before the listing date (or a gap). Step
                # forward a page; stop once past what has been collected so far.
                if candles:
                    break
                since += self.page_size * _DAY_MS
                continue
            for candle in page:
                if start_ms <= candle[0] < stop_ms:
                    candles[int(candle[0])] = candle
            next_since = int(page[-1][0]) + _DAY_MS
            if next_since <= since:  # no progress: the exchange ignores 'since'
                break
            since = next_since
        if not candles:
            continue
        rows = [candles[k] for k in sorted(candles)]
        frames.append(pd.DataFrame({
            "symbol": symbol,
            "date": [dt.datetime.fromtimestamp(r[0] / 1000, dt.UTC).date() for r in rows],
            "open": [r[1] for r in rows],
            "high": [r[2] for r in rows],
            "low": [r[3] for r in rows],
            "close": [r[4] for r in rows],
            "adj_close": [r[4] for r in rows],
            "volume": [r[5] for r in rows],
        }).astype({c: "float64" for c in PRICE_COLUMNS[2:]}))
    return pd.concat(frames, ignore_index=True)[PRICE_COLUMNS] if frames else empty_prices()
```

4.2 Storage and incremental updates

Prices are stored per (symbol, date): open, high, low, close, adjusted close and volume.

- **Upserts.** Writes use `INSERT ... ON CONFLICT DO UPDATE`, in the SQLite or PostgreSQL dialect, so re-downloading is idempotent and picks up revisions.
- **Incremental updates.** A stored symbol is re-fetched from five days before its last date, which corrects recent revisions.
- **Batching.** Symbols are grouped by source and start date.

Code 3: Portable upsert
`nabia/marketdata/store.py:66-81`

```
def upsert_rows(engine: Engine, table: Table, rows: list[dict], keys: Sequence[str], chunk: int = 500)
    ↪ -> None:
    """Insert rows, updating the non-key columns of rows whose keys already exist."""
    if not rows:
        return
    dialect = engine.dialect.name
    if dialect == "sqlite":
        from sqlalchemy.dialects.sqlite import insert
    elif dialect == "postgresql":
        from sqlalchemy.dialects.postgresql import insert
    else:  # pragma: no cover - other databases are not supported yet
        raise NotImplementedError(f"upsert is not implemented for {dialect}")
    with engine.begin() as connection:
        for i in range(0, len(rows), chunk):
            statement = insert(table).values(rows[i : i + chunk])
            updates = {c.name: statement.excluded[c.name] for c in table.columns if c.name not in keys}
            connection.execute(statement.on_conflict_do_update(index_elements=list(keys), set_=updates)
    ↪ )
```

Code 4: Incremental download
`nabia/marketdata/download.py:19-72`

```python
def update_market_data(
    store: MarketDataStore,
    source: MarketDataSource | Mapping[str, MarketDataSource],
    assets: Sequence[Asset],
    start: dt.date,
    end: dt.date | None = None,
    full: bool = False,
    overlap_days: int = DEFAULT_OVERLAP_DAYS,
) -> dict[str, int]:
    """Downloads daily prices for ``assets`` and upserts them into ``store``.

    ``source`` is either one source for every asset, or a mapping from source
    name to source, in which case each asset is fetched from ``asset.source``
    (e.g. ``{"yahoo": YahooFinanceSource(), "crypto": CryptoSource()}``).

    Incremental by default: a symbol already in the store is fetched from
    ``overlap_days`` before its last stored date (so recent revisions are
    picked up); a new symbol from ``start``. ``full=True`` re-downloads
    everything from ``start``. Returns the number of rows written per symbol
    (0 for symbols the source returned no data for).
    """
    sources: dict[str, MarketDataSource]
    if isinstance(source, Mapping):
        sources = dict(source)
        unknown = {a.source for a in assets} - sources.keys()
        if unknown:
            raise ValueError(f"no source given for: {sorted(unknown)}")
        source_of = {a.symbol: a.source for a in assets}
    else:
        sources = {source.name: source}
        source_of = {a.symbol: source.name for a in assets}
    for name in sorted({source_of[a.symbol] for a in assets}):
        store.upsert_assets([a for a in assets if source_of[a.symbol] == name], source=name)
    last = {} if full else store.last_dates()

    # Group symbols by source and fetch start, so each group is one batched request.
    groups: dict[tuple[str, dt.date], list[str]] = {}
    for asset in assets:
        from_date = start
        if asset.symbol in last:
            from_date = max(start, last[asset.symbol] - dt.timedelta(days=overlap_days))
        groups.setdefault((source_of[asset.symbol], from_date), []).append(asset.symbol)

    written = {asset.symbol: 0 for asset in assets}
    for (name, from_date), symbols in sorted(groups.items()):
        log.info("fetching %d symbols from %s since %s", len(symbols), name, from_date)
        prices = sources[name].fetch(symbols, from_date, end)
        for symbol, rows in prices.groupby("symbol"):
            written[str(symbol)] = store.upsert_prices(rows)
            log.info("%s: %d prices stored (%s to %s)", symbol, written[str(symbol)], rows["date"].min
    ↪ (), rows["date"].max())
    for symbol, count in written.items():
        if count == 0:
            log.warning("no data for %s", symbol)
    return written
```

Code

universe — `nabia/marketdata/universe.py:24-70`
Yahoo source — `nabia/marketdata/sources.py:48-67`
loading a price matrix — `nabia/marketdata/store.py:136-168`

5 Analytics on price frames

`nabia.analytics` computes the statistics the agents of the web UI see, and works on any pandas frame of prices (dates by assets).

All statistics use daily close prices up to `as_of`, over `stats_history_days` (400) calendar days. Volatilities are annualized with $\sqrt{252}$, standard deviations are sample standard deviations, and returns are simple returns.

Aligned returns keep the dates where all assets have a price, then take returns. They are used for correlations only. Single-asset statistics use each asset's own history.

Correlation is the Pearson correlation matrix ρ of the last w aligned returns (60 by default, at least 3). The *average correlation* is

$$\bar{\rho} = \frac{1}{n(n-1)} \sum_{i \neq j} \rho_{ij}.$$

Rolling volatility is $\sigma^{(w)} = \mathrm{sd}(r_{T-w+1..T})\sqrt{252}$ for $w \in \{20, 60, 120\}$. The *volatility percentile* is computed from the series v_s of rolling 20-day standard deviations over the last 252 dates:

$$\pi = \frac{1}{|v|} \sum_{s} \mathbf{1}[v_s \leq v_T].$$

So $\pi = 1$ means the highest of the year.

Past returns $P_T / P_{T-n} - 1$ for $n \in \{21, 63, 126, 252\}$ observations (1, 3, 6 and 12 months).

Moving-average distance $P_T / \overline{P}_n - 1$, where $\overline{P}_n$ is the mean of the last $n \in \{50, 200\}$ prices.

Volatility-scaled 12-1 momentum

$$m = \frac{P_{T-21}/P_{T-252} - 1}{\mathrm{sd}(r_{T-251..T})\sqrt{252}}.$$

It is the 11-month return ending one month ago, which skips the short-term reversal, measured in units of the asset's own annual volatility.

Drawdowns over the last 253 prices: $D_t = 1 - P_t / \max_{s \leq t} P_s$. The tools report the current value and the maximum.

Market regime combines five numbers:

- $\bar{\rho}$ over 60 and over 252 aligned returns;
- the median volatility percentile;
- the share of assets above their 200-day average;
- the share of assets with a positive 12-month return.

Code 5: Trend signals
`nabia/analytics.py:51-75`

```python
def trend_signals(prices: pd.DataFrame) -> pd.DataFrame:
    """Per symbol: past returns (1m, 3m, 6m, 12m), distance from the 50- and
    200-day moving averages, a volatility-scaled 12-1 momentum score, and the
    current and 1-year maximum drawdowns."""
    out = {}
    for symbol in prices.columns:
        p = prices[symbol].dropna()
        last = p.iloc[-1]
        row: dict[str, float] = {}
        for label, n in (("ret_1m", 21), ("ret_3m", 63), ("ret_6m", 126), ("ret_12m", 252)):
            row[label] = float(last / p.iloc[-n - 1] - 1) if len(p) > n else float("nan")
        for n in (50, 200):
            row[f"vs_ma{n}"] = float(last / p.tail(n).mean() - 1) if len(p) >= n else float("nan")
        r = p.pct_change().dropna()
```

```
        vol = r.tail(252).std() * np.sqrt(PERIODS_PER_YEAR)
        if len(p) > 252 and vol > 0:
            row["momentum_12_1"] = float((p.iloc[-22] / p.iloc[-253] - 1) / vol)  # skip the last month
        else:
            row["momentum_12_1"] = float("nan")
        year = p.tail(253)
        drawdown = 1 - year / year.cummax()
        row["drawdown"] = float(drawdown.iloc[-1])
        row["max_drawdown_1y"] = float(drawdown.max())
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 6: Volatilities and their percentile
`nabia/analytics.py:36-48`

```
def rolling_volatility(prices: pd.DataFrame, windows: tuple[int, ...] = (20, 60, 120)) -> pd.DataFrame:
    """Per symbol: annualized volatility over each window (its own returns, not
    aligned with the other assets), plus where the 20-day volatility sits in
    its last year of history (percentile, 0..1)."""
    out = {}
    for symbol in prices.columns:
        r = prices[symbol].dropna().pct_change().dropna()
        row = {f"vol_{w}d": float(r.tail(w).std() * np.sqrt(PERIODS_PER_YEAR)) if len(r) >= w else
    ↪ float("nan")
               for w in windows}
        rolling = r.rolling(20).std().dropna().tail(PERIODS_PER_YEAR)
        row["vol_20d_percentile_1y"] = float((rolling <= rolling.iloc[-1]).mean()) if len(rolling) else
    ↪  float("nan")
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 7: Market regime
`nabia/analytics.py:78-91`

```
def market_regime(prices: pd.DataFrame) -> dict[str, float]:
    """Cross-asset summary: average correlation now (60d) vs over the last year,
    median volatility percentile, and the share of assets in an uptrend."""
    corr_now = average_correlation(correlation_matrix(prices, 60))
    corr_year = average_correlation(correlation_matrix(prices, 252)) if len(aligned_returns(prices)) >=
    ↪  60 else float("nan")
    vols = rolling_volatility(prices)
    trends = trend_signals(prices)
    return {
        "avg_correlation_60d": corr_now,
        "avg_correlation_1y": corr_year,
        "median_vol_percentile_1y": float(vols["vol_20d_percentile_1y"].median()),
        "share_above_ma200": float((trends["vs_ma200"] > 0).mean()),
        "share_positive_12m": float((trends["ret_12m"] > 0).mean()),
    }
```

Code

correlation matrix — `nabia/analytics.py:21-26`
average correlation — `nabia/analytics.py:29-33`

6 Worked examples on market data

`examples` has six numbered scripts on `nabia_python_bridge`. They run on daily closes from Yahoo Finance, cached as CSV, with the shared market database (`../data/market.db`) as an offline fallback. A seventh, `risk_budgeting_from_market_data.py`, uses the `nabia` package end to end: download, backtest and the weights to hold now. Run them from `python/` with `.venv/bin/python examples/01_factor_replication.py`, or in PyCharm. Together they use every name the bridge exports. Each script prints its results with comments computed from the numbers, and saves its figures as PNG in `examples/figures/`, opening them when the matplotlib backend is interactive.

Script	Content and figures
`01_factor_replication`	A fund built as 0.5 Nasdaq 100 + 0.3 gold + 0.2 10y notes, plus alpha and 2% noise. The engine recovers 0.497/0.300/0.193 out of 14 candidates, and the bootstrap flags a spurious fourth factor. Figures: lasso path, R^2 by subset size, validation curve, bootstrap intervals, replication out of sample, tracking error
`02_risk_parity_and_budgeting`	Equal weights against risk parity for volatility, VaR and ES, inverse volatility, and a 40/40/20 class budget. Figures: capital against risk shares, correlations, risk by measure
`03_portfolio_backtest`	Walk-forward risk parity variants against equal weights, and a budget tilt in 2022. Figures: equity and drawdowns, weights and risk shares through time, leverage under a volatility target
`04_signals_and_stats`	Fat tails and volatility clustering, smoothers, the band-pass cycle, EWMA crossovers. Figures: density against the normal, tail ratios, autocorrelations, trend filters and rule equity
`05_forecasting`	Kalman filters and SES on realized volatility, a local linear trend on gold, and Gaussian processes on natural gas, all scored on a holdout. Figures: forecasts with intervals, innovations, slope, kernels, Gram matrix
`06_optimization`	Max-Sharpe with L-BFGS against the closed form, constrained mean-variance with AL and SQP, GARCH(1,1) with Nelder–Mead, and rule calibration with differential evolution. Figures: efficient frontier, GARCH volatility, objective surface with the DE optima

Code 8: Every exported name must appear in an example
`tests/bridge/test_examples_coverage.py:12-35`

```
def unused_names() -> list[str]:
    text = "\n".join(p.read_text() for p in EXAMPLES.glob("0*.py"))
    words = set(re.findall(r"[A-Za-z_][A-Za-z0-9_]*", text))
    missing = []
    for modname in MODULES:
        mod = getattr(nb, modname)
        for name in mod.__all__:
            obj = getattr(mod, name)
            if name not in words:
                missing.append(f"{modname}.{name}")
            if not (isinstance(obj, type) and obj.__module__.startswith("nabia_python_bridge._core")):
                continue
            for member in vars(obj):
                if member in ("__add__", "__mul__"):
                    used = re.search(r"Kernel\([^)]*\)\s*[+*]|\bse \+|\bper \*", text) is not None
                elif member == "__call__":
                    used = re.search(r"\b(k|se|per|kernel)\(", text) is not None
                elif member.startswith("_") or member in ("name", "value"):
                    continue
                else:
                    used = member in words
                if not used:
                    missing.append(f"{modname}.{name}.{member}")
    return missing
```

Code

data download, cache and fallback — `examples/_data.py:49-67`
figure saving — `examples/_plots.py:49-57`

Part IV
Verification and limitations

7 Verification

Bridge (`tests/bridge`, 34 tests): every module of `nabia_python_bridge` against closed forms and numpy, the binding semantics, and the check that the examples use the whole exported API. The C++ project's wheel scripts run them on every wheel.

Package (`tests`, 15 tests): `nabia.backtest` against the command-line application on its own example (a copy in `tests/data`), schedules and errors; the market data store, sources and incremental download against fakes. Two opt-in tests (`NABIA_NETWORK_TESTS=1`) download live data.

This document Its excerpts and code locations are extracted at build time.

8 Limitations

- **Continuous futures** jump at rolls. Back-adjusted series would remove spurious returns.
- **Asynchronous closes** (Asia against US) bias same-day correlations downwards. Multi-day returns or synchronized prices would reduce the bias.
- **The wheel** must match the interpreter of `.venv`; build it for that Python version first.